

3

Date :

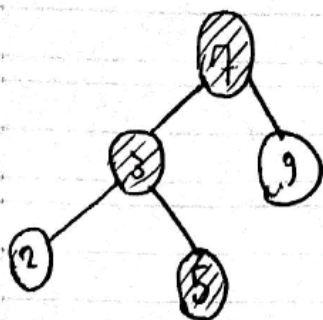
1

Page No.

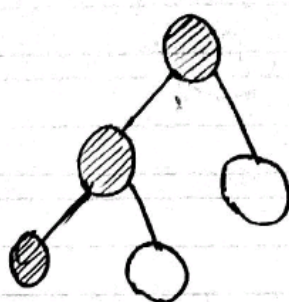
Subject :

Top algorithms you should know

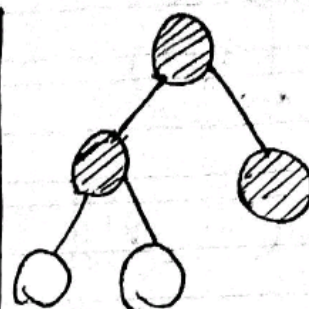
* Search Algorithms:



Binary Search

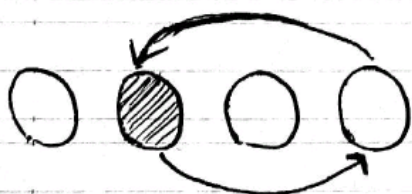


Depth-First Search (DFS)



Breadth-First Search (BFS)

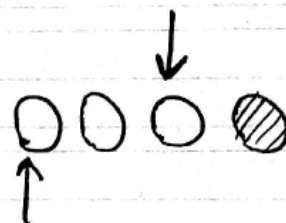
* Sorting Algorithms:



Insertion Sort

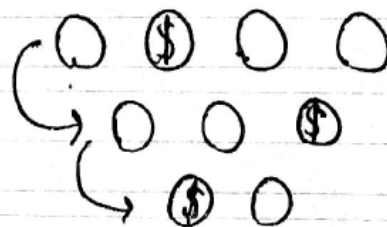


Merge Sort



Quick Sort

* Greedy Algorithms:



Date: 2 / /



Subject:

Binary Search:

(sorted list)

1, 2, 3, 4, 5, 6, 7, 8, ^{50?} 100

26

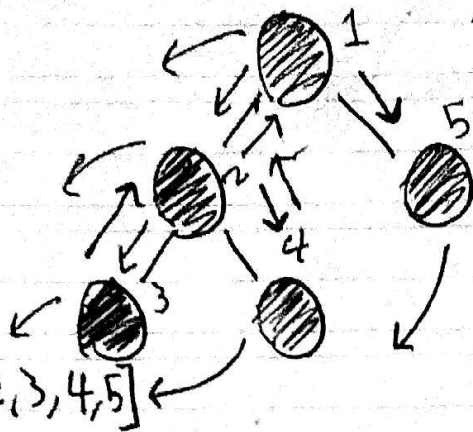
50?

LOWER

time comp: $O(\log_2(n))$

worst case/ algo	linear search	binary search
$n=10000$	10000 ($O(n)$)	14 ($O(\log_2(n))$)

(DFS) :



① start at root node and go as far as possible to the end

② when you hit the deepest point you go back to an unvisited branch (backtracking)

Uses: Solve a maze:

(you continue all the way long and if you find a wall back track to another way till the end)



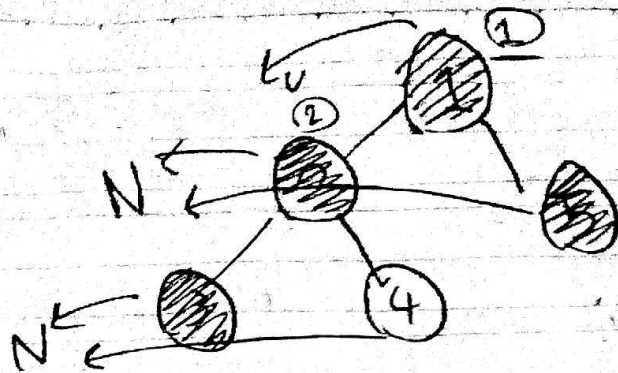
time cons $O(V+E)$

Vertices/
Nodes

edges/
branches

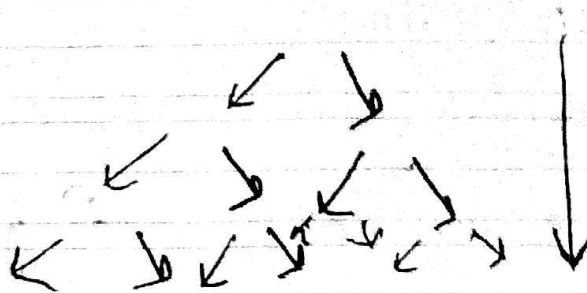
Sabbagh

(BFS)

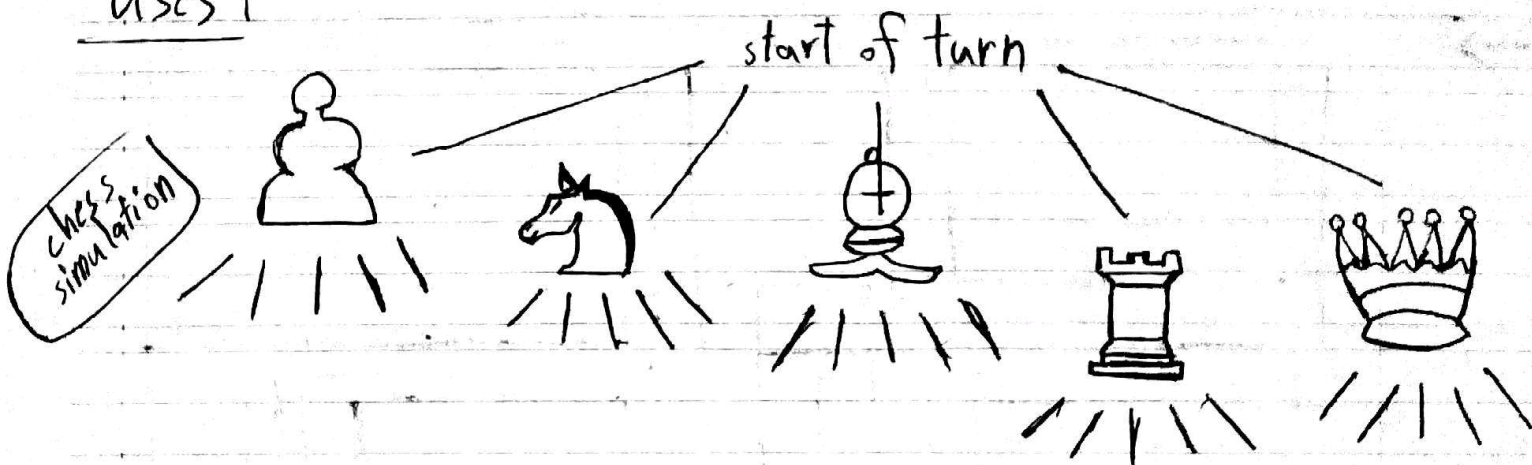


neighbours = [2, 3, 4]

visited = [1, 2, 3]



Uses :



5

Insertion Sort:

7 3 1 8 2 4 5 6

look at the 1st compare with the 2nd if ~~the~~ ^{2nd} is less than ~~the~~ ^{1st} swap and continue and if the 3rd is less than the 2nd swap between them and after swap if the 2nd is less than the 1st swap and continue if 4th is less than third swap and do the chain of swapping and so on.

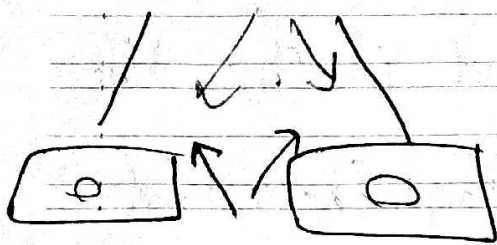
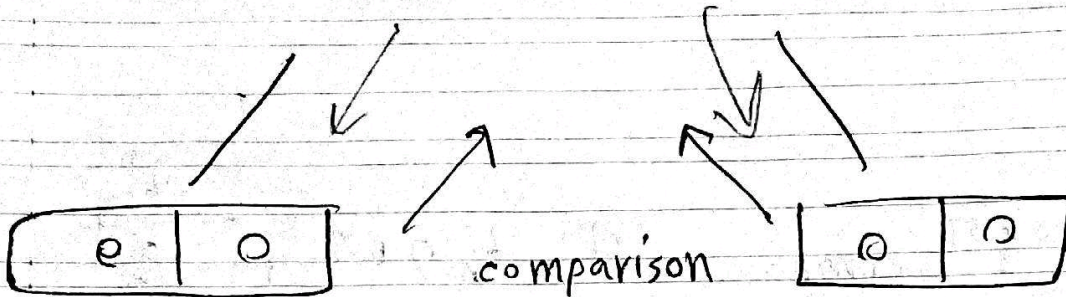
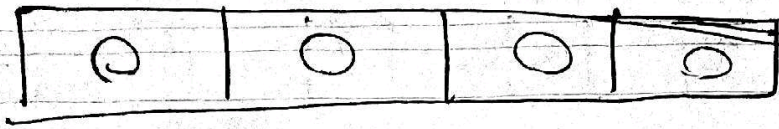
best case time complexity: $O(n)$ already sorted (mostly)

average case time complexity:

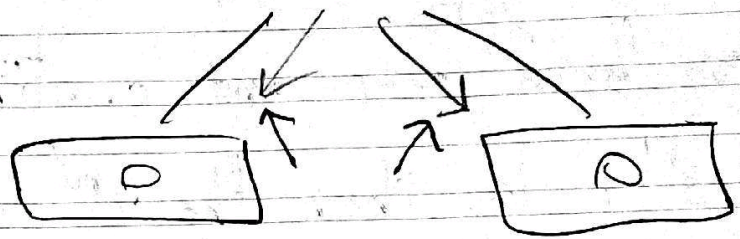
worst case time complexity: $O(n^2)$ nothing is sorted

good if sorted or small.

Merge Sort (Divide-and-conquer) (recursive)



comparison
(swap if 2nd is greater)



comparison

best case time complexity: $O(n \log n)$

average case time complexity: $O(n \log n)$

worst case time complexity: $O(n \log n)$

smaller & sorted | larger & unsorted

insertion $O(n)$

$O(n^2)$

merge $O(n \log n)$

$O(n \log n)$

Quick Sort: (Divide-and-conquer) (recursive)

pick a pivot the ~~put it in its place~~
 and move it to the end and make two pointers
 one on 0 and on length-1, compare if the left
 is greater and right is smaller swap, continue
 until our pointers cross the place the pivot
 in the pointers position. (less)

repeat on the list at the left of the pivot
 and the list at the ^(greater) right of the pivot

$$O(n \log n)$$

$$O(n^2)$$

average case: fastest (2 to 3 times faster)
 + than merge

merge	quick
$O(n)$	$O(n \log n)$



Date

(8)



Subject

Greedy Algorithms:

algorithms that just looks for how
like (traveller salesman)